TraceBack

HACKATHON JURY ROUND

20 Judge Questions — Answered

Ready-to-deliver answers for the SIH jury round, grounded in what TraceBack — the Lost & Found Item Tracker — actually is and does. Each answer includes a one-line version to say out loud, and honest notes where something is a known limitation.

Node + Express + SQLite | React + Vite | 5-signal matching engine | Perceptual image hashing | No paid third-party APIs | 44 automated e2e checks

PROBLEM, USERS & UNIQUENESS

1 What exact problem are you solving, and why is it important?

On any campus, office or public place, lost and found items are handled through scattered notice boards, WhatsApp groups and a lost-and-found desk that never talk to each other. An owner and a finder can be metres apart and never connect. **TraceBack gives both sides one place to report, automatically matches lost against found, verifies the real owner, and tracks the item to a confirmed return.**

It matters because it saves people time and money (fewer replacement purchases), reduces daily friction on every campus, and turns an unreliable manual process into a trustworthy, trackable one.

SAY THIS **"We replace scattered notice boards and chat groups with one system that actively matches lost and found items and proves ownership before returning them."**

2 Who is your target user, and how did you validate this is a real problem?

Primary users are students and staff on a campus; the same design extends to offices, transit hubs and event venues. Validation is that this is a universally recognised pain — every college has an overflowing lost-and-found box and a daily stream of "has anyone seen my..." messages, and the problem statement itself confirms institutional demand.

Honest note: our validation so far is observational rather than a formal survey — the first thing we'd do post-hackathon is a one-campus pilot to measure the real return rate.

SAY THIS **"It's a problem on literally every campus — the current 'system' is a cardboard box and a WhatsApp group. A pilot is our next step to quantify it."**

3 What is innovative or unique about your solution?

- It **actively matches** lost against found instead of just listing items — a transparent 5-signal scoring engine, not a black box.
- **Real perceptual image similarity** computed locally, so photos are compared even when text differs.
- A **privacy-first ownership-verification flow**: private questions only the true owner can answer, auto-scored, with identities masked until approval.
- Every match is **explainable** — it shows the per-factor breakdown and the reasons behind the score.

4 How is your solution different from existing solutions?

- **Notice boards / WhatsApp groups:** no matching, no verification, no tracking, and phone numbers are public.
- **Generic classifieds:** listing, not matching — the user still searches manually.
- **Basic college portals:** simple forms that rarely verify ownership or track status.

TraceBack is the only approach that **matches + verifies + tracks + keeps identities private**, end to end.

TECHNOLOGY, ARCHITECTURE & BUILD

5 Why did you choose this technology stack, architecture, or algorithm?

Stack: Node.js + Express + React is a widely known, well-supported stack that's easy to build and extend under time pressure. **Database:** SQLite for zero-setup reliability during a demo — every query is plain SQL, so moving to PostgreSQL later is a driver swap, not a rewrite. **Algorithm:** a transparent weighted-scoring model rather than deep learning, because a verification-critical workflow needs explainability and must run offline with no GPU or paid API.

Architecture: a single Node service is easy to run and reason about; the code is already split into clean modules that could each become a separate service later.

6 Explain your architecture — walk me from input to output.

- A user files a report in the React client (title, category, description, location, date, photo).
- The client calls the REST API with a JWT; Express validates the token and the input.
- The report is written to SQLite; the photo gets a 64-bit perceptual hash.
- The **matching engine runs immediately** against all open opposite-type reports and computes a five-factor score for each.
- Matches above the threshold are saved and **notifications are created for both users**.
- The owner claims the item › answers private questions (auto-scored) › the finder/admin approves › identities unlock › handover › status becomes **RETURNED › CLOSED**.

7 Which parts did you actually build, and which are simulated or third-party?

Built and real (all persisted in the database): authentication, lost/found reporting with photo upload, the matching engine, perceptual image hashing, verification scoring, the full claim workflow, item-scoped chat, notifications, the admin console and analytics.

Simulated / limited, stated honestly: the demo uses a seeded dataset with generated placeholder photos; notifications use short-interval polling rather than real-time push; there is no email/SMS verification yet; uploads are stored on local disk rather than cloud storage.

SAY THIS **"There are no paid third-party services anywhere — the whole system, including image similarity, runs on our own code. What's not production-grade yet is infra hardening, not the core logic."**

8 What was the hardest technical challenge, and how did you solve it?

Making text matching meaningful when two people describe the **same object with completely different words** — the owner lists what's inside ("ID card, ₹300"), the finder describes what's visible ("black wallet on a canteen table"). We solved it with a synonym lexicon plus a blend of token overlap, Jaccard and character-bigram cosine, then a **calibration curve** — because independently written descriptions of the same object share only 30–45% of their words, so a raw 40% overlap must read as a strong signal.

A second challenge was cross-platform install: a native database module failed to compile on Windows without a C++ toolchain. We fixed it by auto-selecting Node's built-in `node:sqlite`, so the app installs with no compiler.

9 Why do you need AI/ML? Could a simpler rule-based approach work?

We deliberately **didn't** reach for heavy ML. Our engine is essentially smart, explainable heuristics — weighted similarity plus perceptual image hashing. A naive exact-name rule fails instantly ("phone" vs "Redmi Note 12 with cracked guard"), so we needed similarity and synonyms, but not a black box.

This is a strength for a verification workflow: every score is fully explainable, which a deep model isn't. Sentence-transformer embeddings are a clean upgrade path we can drop in later without changing the API.

SAY THIS **"We use the minimum intelligence that solves the problem well and stays explainable — that's the right call when the output gates who gets someone's property back."**

10 What data are you using, where is it from, and how reliable is it?

The data is **user-generated reports** — title, category, description, location, date and an optional photo. No external datasets are required. Its reliability naturally varies by user, which is exactly why we never trust a single field: we combine five independent signals and add a mandatory human verification step. Locations use a controlled vocabulary to stay consistent, and the demo runs on a seeded, realistic campus dataset.

ACCURACY, ROBUSTNESS & LIMITATIONS

11 How do you measure accuracy, performance, or effectiveness?

- **Match quality:** the admin analytics track match success rate (confirmed ÷ generated), claim approval rate and average resolution time.
- **Verification separation:** in testing a genuine owner scores ~**89%** while an impostor scores ~**2%** — a wide, usable gap.
- **Correctness:** a **44-assertion end-to-end test** drives the whole journey, plus an automated screenshot pass over every screen.
- **Real-world:** we'd measure the return rate during a campus pilot.

12 What happens with incorrect, unexpected, or malicious input?

- Server-side validation on every write: report type, required fields, valid dates, a category whitelist, and file-type/size limits on uploads.
- Auth required on protected routes (401) and role checks on admin routes (403); you cannot claim your own item.
- All SQL uses **parameterised prepared statements**, so injection isn't possible; weak matches below the threshold are hidden; verification answers are compared, never trusted.

Honest gap: we don't have request rate-limiting yet — it's on the production checklist.

13 What is the biggest limitation or weakness of your current solution?

Text matching is similarity-based, not true semantic understanding, so unusual phrasing can under-score — mitigated by the human review step and by re-scanning. Beyond that: notifications poll rather than push in real time, the image hash catches visual similarity but not fine detail, and there's no email verification yet, so accounts are trust-based. All are known, with clear next steps.

SAY THIS **"Our biggest limitation is semantic text understanding — and that's exactly why a human always makes the final call before an item changes hands."**

14 What are the biggest security and privacy risks, and how do you address them?

Risks: someone falsely claiming an item, exposure of users' contact details, and account abuse.

- Passwords are **bcrypt-hashed**; sessions are stateless **JWTs**.
- Names and emails are **masked** in public responses; phone numbers are never sent by the API.
- Private verification answers are **never returned** by any endpoint.
- Identities are revealed **only after a claim is approved**; access is role-based; SQL is parameterised.

Next: rate limiting, email/college-ID verification, and an audit log.

SCALE, DEPENDENCIES & THE ROAD TO PRODUCTION

15 It works for a few users. What about 100,000 or 1 million?

The design scales without a rewrite: swap SQLite for **PostgreSQL** (plain SQL, so it's a driver change), run the **stateless API behind a load balancer** with horizontal replicas, move uploads to **object storage** (S3), and cache hot reads. The matching step is the one thing to re-engineer — we'd pre-filter candidates and move scoring to a background worker queue (see Q16).

16 What is the biggest performance or scalability bottleneck?

The matching step: each new report is scored against all open opposite reports. That's fine for a campus, but at massive scale it's the hotspot. The fix is straightforward — **pre-filter candidates by category + location + date window** using indexes we already have, then run the scoring **asynchronously in a worker queue** so the user's request returns instantly and matches arrive as notifications.

17 What if an external API, AI model, or database becomes unavailable?

This is a real advantage of our design: **TraceBack has no external API or third-party model dependency** — image hashing and matching run locally, so there's nothing external to fail. The image library is even optional: if it's missing, image similarity is simply skipped and the other four weights re-normalise, so the app keeps working. If the database is briefly unavailable, the API returns clean error responses instead of crashing.

SAY THIS **"There's no external service to go down — our intelligence is self-contained. That's a deployment and reliability win."**

18 What is the gap between your prototype and a production-ready system?

The core workflow, data model and matching engine are already production-shaped — the gap is **hardening and infrastructure, not logic**: email/college-ID verification, rate limiting and abuse protection, PostgreSQL + object storage, WebSocket real-time push, monitoring and logging, automated CI, and a mobile app.

19 If you had one more month, what would you improve first and why?

Real-time notifications (WebSockets) and a mobile-first PWA — because adoption depends on instant alerts and phone-first use. Then **map-based location matching** (haversine distance on coordinates) to sharpen the strongest signal, and a **one-campus pilot** to gather real return-rate data to guide everything after.

20 Why should the judges choose your solution over the others?

Because it's a **complete, working, end-to-end system** — not a mockup — verified by 44 automated checks. It solves a **real, universal problem** with an **explainable** matching engine, **genuine** image similarity, and a **privacy-first verification flow that actually prevents false claims**. It runs anywhere with zero setup and **no paid dependencies**, the architecture scales cleanly, and it's polished across 23 screens — and we're honest about exactly where its limits are.

SAY THIS **"It works today, it's explainable, it protects users, and it costs nothing to run — a solution you could pilot on this campus next week."**